



MOBILE APPLICATION DEVELOPMENT

ANÁLISE E DESENVOLVIMENTO DE SISTEMAS



AULA 04

**Mantendo a interface
consistente**

Como reagir a falhas e atualizar dados após uma mudança?



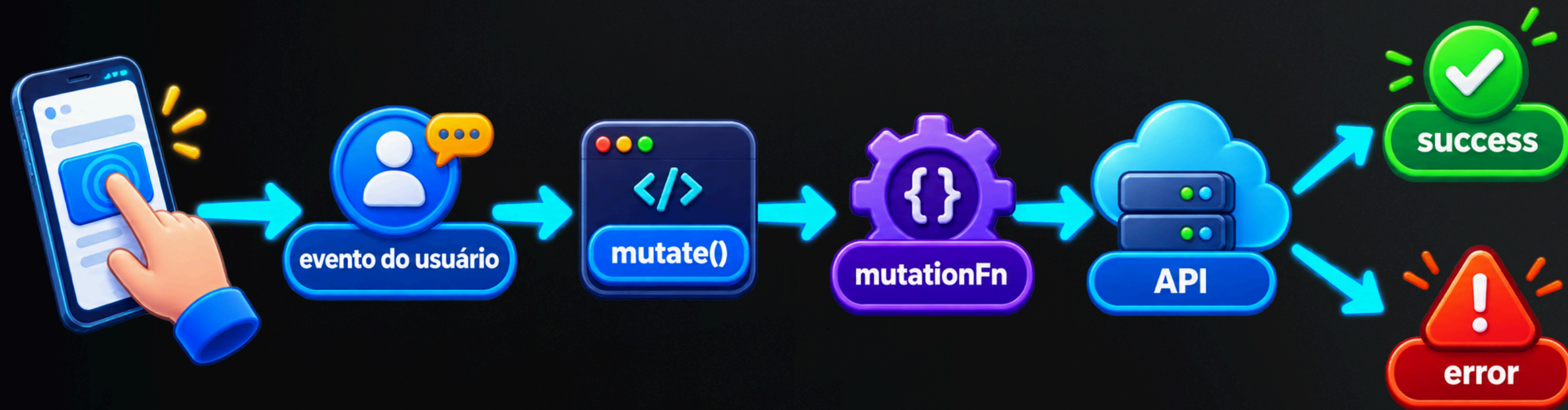
“*Tudo o que pode dar errado, dará errado.*”
— *Lei de Murphy*





Mentimeter

O MODELO QUE CONSTRUÍMOS



A OPERAÇÃO TERMINOU. E AGORA?

ERROR

A falha pode ser temporária.

Talvez seja possível tentar novamente.

SUCCESS

A API mudou.

O cache pode continuar com os dados anteriores.

QUANDO UMA OPERAÇÃO FALHA

RETRY

NEM TODA FALHA É DEFINITIVA

Uma requisição pode falhar porque:

- a conexão oscilou;
- o servidor ficou indisponível por alguns segundos;
- a requisição excedeu o tempo esperado.

UMA NOVA TENTATIVA PODE FUNCIONAR.

RETRY

Retry é uma nova tentativa da mesma operação depois de uma falha.



QUERY E MUTATION COMEÇAM DE FORMAS DIFERENTES

Operação	Comportamento padrão
Query	Tenta novamente até 3 vezes no cliente
Mutation	Não tenta novamente

A DIFERENÇA EXISTE PORQUE REPETIR UMA MUDANÇA PODE PRODUZIR NOVOS EFEITOS.

CONFIGURANDO O RETRY DE UMA QUERY

```
const tweetsQuery = useQuery({  
  queryKey: ["tweets"],  
  queryFn: fetchTweets,  
  retry: 2,  
});
```

A configuração agora representa:

1 tentativa inicial + 2 novas tentativas

REPETIR NÃO CORRIGE QUALQUER ERRO

PODE AJUDAR

- conexão instável
- timeout
- servidor temporariamente indisponível

NÃO RESOLVE

- endereço incorreto
- dados inválidos
- acesso não autorizado

E EM UMA MUTATION?

```
const createTweetMutation = useMutation({  
  mutationFn: createTweet,  
  retry: 2,  
});
```

O CÓDIGO PERMITE. A OPERAÇÃO PRECISA PERMITIR TAMBÉM.

REPETIR UMA MUDANÇA PODE DUPLICÁ-LA



O CLIENTE VIU UMA FALHA. O SERVIDOR PODE TER CONCLUÍDO A OPERAÇÃO.

ANTES DE REPETIR, AVALIE O EFEITO

Operação	O que pode acontecer ao repetir?
Consultar filmes	A mesma lista é consultada novamente
Atualizar o nome para “Ana”	O mesmo valor é enviado novamente
Criar um pedido	Outro pedido pode ser criado
Confirmar um pagamento	Uma nova cobrança pode ser solicitada

OPERAÇÕES IDEMPOTENTES

Uma operação é idempotente quando executá-la várias vezes produz o mesmo efeito no estado do servidor que executá-la uma única vez.

```
executar uma vez    → estado final  
executar novamente → mesmo estado final
```

RETRY EM MUTATIONS EXIGE CONHECER O CONTRATO DA API.

PARE E DECIDA

Para cada operação, responda:

retry automático
retry com cautela
sem retry

1. Carregar a lista de filmes.

2. Atualizar o nome de um filme.

3. Criar um pedido.

QUANDO UMA MUTATION FUNCIONA

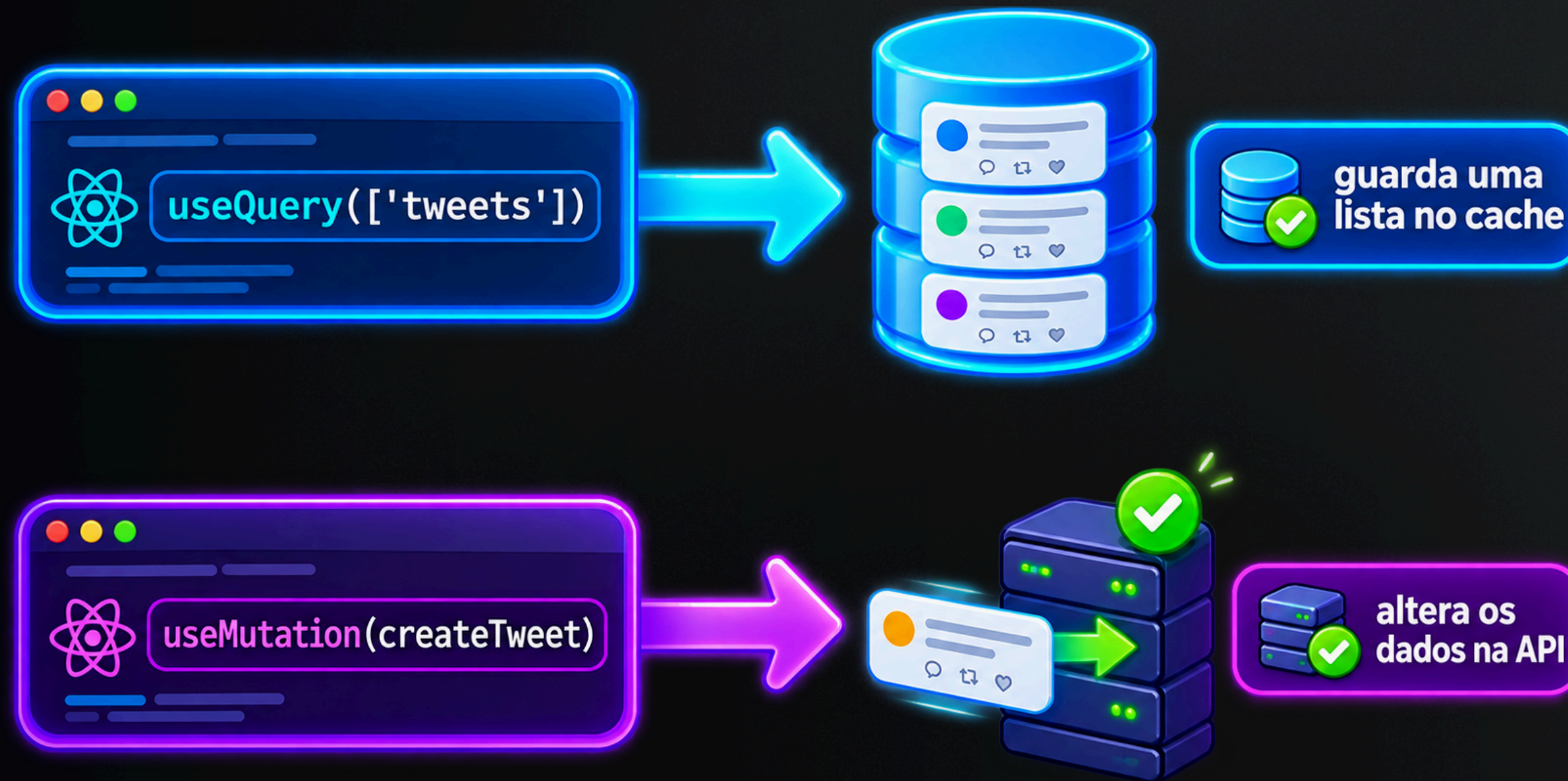
CACHE INVALIDATION

A API MUDOU. A TELA NÃO.



A MUTATION FUNCIONOU. POR QUE A TELA ESTÁ DESATUALIZADA?

QUERY E MUTATION SÃO INDEPENDENTES



A MUTATION NÃO DESCOBRE AUTOMATICAMENTE QUAIS QUERIES FORAM AFETADAS.

INVALIDAR É DECLARAR QUE O DADO ENVELHECEU



✗ invalidar $\neq$ inserir o novo item manualmente



✗ invalidar $\neq$ apagar definitivamente o cache



✓ invalidar $=$ marcar os dados como desatualizados

O QUE ACONTECE AO INVALIDAR UMA QUERY?

1. A query é marcada como `stale`.
2. Se estiver ativa, uma nova busca acontece em segundo plano.

cache antigo → stale → refetch → cache atualizado

ACESSANDO O QUERYCLIENT

```
import { useQueryClient } from "@tanstack/react-query";  
  
const queryClient = useQueryClient();
```

O QUERYCLIENT PERMITE INTERAGIR COM AS QUERIES ARMazenADAS NO CACHE.

INVALIDANDO UMA QUERY

```
queryClient.invalidateQueries({  
  queryKey: ["tweets"],  
});
```

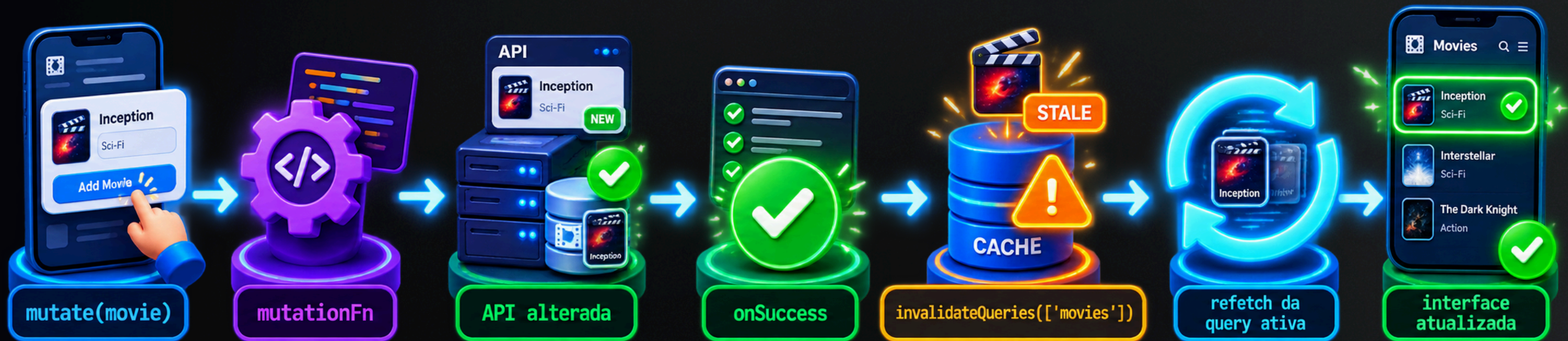
A QUERYKEY IDENTIFICA QUAIS DADOS DEIXARAM DE SER CONFIÁVEIS.

A INVALIDAÇÃO ACONTECE DEPOIS DO SUCESSO

```
const queryClient = useQueryClient();

const createTweetMutation = useMutation({
  mutationFn: createTweet,
  onSuccess: () => {
    queryClient.invalidateQueries({
      queryKey: ["tweets"],
    });
  },
});
```

DA MUTATION À TELA ATUALIZADA



A CHAVE PRECISA REPRESENTAR O DADO AFETADO

```
// Query que carrega a lista
const tweetsQuery = useQuery({
  queryKey: ["tweets"],
  queryFn: fetchTweets,
});

// Invalidação depois de criar um tweet
queryClient.invalidateQueries({
  queryKey: ["tweets"],
});
```

SE AS CHAVES NÃO CORRESPONDEM, A LISTA NÃO É INVALIDADA.

RETRY, REFETCH E INVALIDAÇÃO

Conceito	Por que acontece?	O que faz?
Retry	Uma operação falhou	Repete a operação
Refetch	Os dados precisam ser consultados novamente	Executa outra busca
Invalidação	O cache deixou de ser confiável	Marca a query como <code>stale</code> e pode provocar refetch



CODELAB





DESAFIO



PROBLEMAS COMUNS

- Ativar retry em qualquer mutation.
- Esperar que retry corrija dados inválidos.
- Esperar que a mutation atualize todas as queries.
- Invalidar a query antes de a API concluir a mudança.
- Usar uma `queryKey` diferente da query afetada.
- Confundir invalidação com exclusão do cache.

SÍNTESE



REFERÊNCIAS

- TanStack Query. **Query Retries.**
<https://tanstack.com/query/latest/docs/framework/react/guides/query-retries>
- TanStack Query. **Mutations.**
<https://tanstack.com/query/latest/docs/framework/react/guides/mutations>
- TanStack Query. **Query Invalidation.**
<https://tanstack.com/query/latest/docs/framework/react/guides/query-invalidation>
- TanStack Query. **Invalidations from Mutations.**
<https://tanstack.com/query/latest/docs/framework/react/guides/invalidations-from-mutations>

RECOMENDAÇÃO DE LEITURA

Sistemas confiáveis precisam lidar com falhas, operações repetidas e cópias de dados que deixam de estar sincronizadas.

Projetando Aplicações com Uso Intensivo de Dados – Martin Kleppmann

✨ Por que ler?

- ⚠️ Explica por que sistemas podem falhar mesmo quando parte da operação foi concluída
- ↺ Ajuda a entender os riscos de repetir operações e produzir efeitos duplicados
- 🔁 Discute o desafio de manter diferentes cópias dos dados sincronizadas
- 🧠 Desenvolve o raciocínio sobre confiabilidade, consistência e desempenho além de uma biblioteca específica



LICENÇA

Este material é disponibilizado sob a licença Creative Commons Atribuição 4.0 Internacional (CC BY 4.0).

  Atribuição 4.0 Internacional

Autor: Felipe Cavalcante Vilella de Oliveira, 2026